BVCS — A Bash Version Control System

Offline 1 | Bash Scripting | **Deadline: 10 July 2026, 11:45 pm**

Build **BVCS**, a simplified version control system as a single Bash script. Submit one file named *<your id>.sh* with shebang. Your output must match this spec *exactly*.

> **Constraint:** Use only standard POSIX/GNU utilities (`bash`, `find`, `diff`, `grep`, etc.). Do *not* invoke `git` or any other VCS tool. **Do not copy. Penalty: -100%**

Repository Layout

`bvcs init` creates `.bvcs/` in the current directory with the following structure. Every subcommand except `init` and `help` must verify `.bvcs/` exists first.

```
.bvcs/
  objects/          # one subdirectory per commit
    0001/
      files/        # complete snapshot of every tracked file at this commit
      message       # commit message (one plain-text line)
      timestamp     # commit time   (one plain-text line, YYYY-MM-DD HH:MM:SS)
    0002/ ...
  staging           # staged file paths, one per line, no duplicates
  log               # pipe-delimited history: id|timestamp|message
  HEAD              # commit ID of the most recent commit (empty if none)
```

Commit IDs are sequential 4-digit zero-padded integers (`0001`, `0002`, ...). Each snapshot is *complete*: when committing, copy the previous `files/` tree forward before overwriting with staged files, so `diff`/`status`/`restore` always compare against HEAD directly.

Subcommands

init

```
$ bvcs init
Initialized empty BVCS repository.
```

Error if `.bvcs/` already exists: `Error: BVCS repository already exists.`
Error if not in a repo (all other commands): `Error: Not a BVCS repository. Run 'init' first.`
Error for unknown subcommand: `Error: Unknown subcommand '<name>'.` (exit 1)

add

```
$ bvcs add main.c util.h
Staged: main.c
Staged: util.h
```

- No arguments: `Error: No files specified.`
- File not found: `Error: '<file>' not found.` (skip; continue with remaining)
- Already staged: `Already staged: <file>` (skip; continue)

status

```
$ bvcs status
Staged for commit:          <- files in .bvcs/staging
  main.c
  util.h

Modified (not staged):      <- tracked files not staged whose content differs from HEAD
  readme.txt

Untracked files:            <- working-dir files neither staged nor in HEAD snapshot
  build/output.o
  notes.md
```

Omit any empty category. If all three are empty: `Nothing to commit, working tree clean.`
Always exclude `.bvcs/`. Staged files: order of `.bvcs/staging`. Modified & untracked: alphabetical. Each non-empty category is followed by exactly one blank line, including the last.

commit

```
$ bvcs commit -m "Add utility functions"
[0002] Add utility functions
2 file(s) committed.
```

- No `-m` or empty message: `Error: Commit message required. Use -m "message".`
- Staging area empty: `Error: Nothing to commit.`

log

```
$ bvcs log
commit 0002
Date:     2026-06-26 11:30:00
Message:  Add utility functions

commit 0001
Date:     2026-06-26 10:00:00
Message:  Initial commit
```

Most recent commit first. Blank line after every entry, including the last.
No commits yet: `No commits yet.`

diff

With a file argument: diff that file against HEAD snapshot.
Without: diff every tracked file (show all; none omitted).

```
$ bvcs diff main.c
--- .bvcs/objects/0002/files/main.c
+++ main.c
@@ -1,4 +1,6 @@
 #include <stdio.h>
+#include <stdlib.h>
 ...
```

Use `diff -u -label "<snap-path>" -label "<file>"` to suppress timestamps.
No differences: `<file>: no changes.`

- No commits: `Error: No commits yet.`
- File not tracked: `Error: '<file>' is not tracked.`

restore

```
$ bvcs restore main.c
Restore 'main.c' from commit 0002? [y/N]: y
Restored: main.c
```

Any answer other than `y` or `Y`: print `Aborted.` and exit 0.

- No argument: `Error: No file specified.`
- No commits: `Error: No commits yet.`
- File not in snapshot: `Error: '<file>' not found in commit <id>.`

help

Print a concise usage message listing all subcommands and their syntax. Format is your choice.